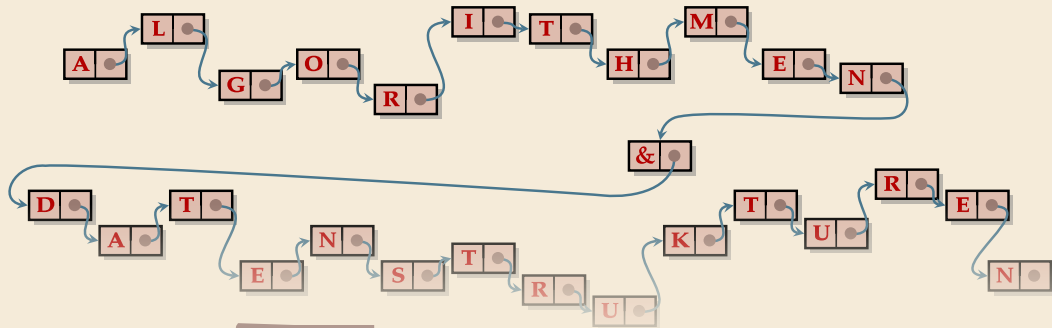




7

Sortieren

Algorithmen & Datenstrukturen · Sommersemester 2026

Prof. Dr. Sebastian Wild

7 Sortieren

- 7.1 Was bedeutet sortiert?
- 7.2 Primitive Sortieralgorithmen
- 7.3 Analyse der primitiven Verfahren
- 7.4 Mergesort
- 7.5 Quicksort
- 7.6 Untere Schranke
- 7.7 Priority Queues & Heapsort
- 7.8 System sorts

7.1 Was bedeutet sortiert?

Sortieren

Gegeben: Array $A[0..n)$ von n Objekten

Ziel: Array so permutieren, dass Objekte aufsteigend sortiert sind.

Sortieren

Gegeben: Array $A[0..n)$ von n Objekten

Ziel: Array so permutieren, dass Objekte aufsteigend sortiert sind.

Beispiele:

- ▶ Hochschulen in Deutschland nach Gründungsjahr

https://de.wikipedia.org/wiki/Liste_der_Hochschulen_in_Deutschland

Sortieren

Gegeben: Array $A[0..n)$ von n Objekten

Ziel: Array so permutieren, dass Objekte aufsteigend sortiert sind.

Beispiele:

- ▶ Hochschulen in Deutschland nach Gründungsjahr
https://de.wikipedia.org/wiki/Liste_der_Hochschulen_in_Deutschland
- ▶ Züge nach Abfahrtszeit
<https://www.bahnhof.de/marburg-lahn/abfahrt>
- ▶ Index in Büchern
- ▶ Arbeitsgruppen des FB12, alphabetisch nach Nachnamen der Professor:innen
<https://www.uni-marburg.de/de/fb12/arbeitsgruppen>
- ▶ ...

Sortieren

Gegeben: Array $A[0..n)$ von n Objekten, Sortierkriterium

Ziel: Array so permutieren, dass Objekte aufsteigend sortiert sind.

Beispiele:

- ▶ Hochschulen in Deutschland nach Gründungsjahr
https://de.wikipedia.org/wiki/Liste_der_Hochschulen_in_Deutschland
- ▶ Züge nach Abfahrtszeit
<https://www.bahnhof.de/marburg-lahn/abfahrt>
- ▶ Index in Büchern
- ▶ Arbeitsgruppen des FB12, alphabetisch nach Nachnamen der Professor:innen
<https://www.uni-marburg.de/de/fb12/arbeitsgruppen>
- ▶ ...

Typischer Fall: verschiedene Sortierkriterien denkbar/möglich
 $\rightsquigarrow$ muss mit angegeben werden

Sample sort client 1

Goal. Sort **any** type of data.

Ex 1. Sort random real numbers in ascending order.

 seems artificial (stay tuned for an application)

```
public class Experiment
{
    public static void main(String[] args)
    {
        int N = Integer.parseInt(args[0]);
        Double[] a = new Double[N];
        for (int i = 0; i < N; i++)
            a[i] = StdRandom.uniform();
        Insertion.sort(a);
        for (int i = 0; i < N; i++)
            StdOut.println(a[i]);
    }
}
```

```
% java Experiment 10
0.08614716385210452
0.09054270895414829
0.10708746304898642
0.21166190071646818
0.363292849257276
0.460954145685913
0.5340026311350087
0.7216129793703496
0.9003500354411443
0.9293994908845686
```


Sample sort client 2

Goal. Sort **any** type of data.

Ex 2. Sort strings in alphabetical order.

```
public class StringSorter
{
    public static void main(String[] args)
    {
        String[] a = StdIn.readAllStrings();
        Insertion.sort(a);
        for (int i = 0; i < a.length; i++)
            StdOut.println(a[i]);
    }
}
```

```
% more words3.txt
bed bug dad yet zoo ... all bad yes
```

```
% java StringSorter < words3.txt
all bad bed bug dad ... yes yet zoo
[suppressing newlines]
```

Sample sort client 3

Goal. Sort **any** type of data.

Ex 3. Sort the files in a given directory by filename.

```
import java.io.File;

public class FileSorter
{
    public static void main(String[] args)
    {
        File directory = new File(args[0]);
        File[] files = directory.listFiles();
        Insertion.sort(files);
        for (int i = 0; i < files.length; i++)
            StdOut.println(files[i].getName());
    }
}
```

```
% java FileSorter .
Insertion.class
Insertion.java
InsertionX.class
InsertionX.java
Selection.class
Selection.java
Shell.class
Shell.java
ShellX.class
ShellX.java
```

Totale Ordnung

Damit Sortieren wohldefiniert ist, benötigen wir ein sinnvolles Sortierkriterium.

Informell: Alle Objekte sind vergleichbar und Vergleiche ergeben keine Widersprüche.

Totale Ordnung

Damit Sortieren wohldefiniert ist, benötigen wir ein sinnvolles Sortierkriterium.

Informell: Alle Objekte sind vergleichbar und Vergleiche ergeben keine Widersprüche.

Formal: Für eine Menge U (Universum) ist eine Relation $\leq \subset U \times U$ eine *totale Ordnung*, falls folgende Bedingungen erfüllt sind

$$(x, y) \in \leq \quad x \leq y$$

1. **Reflexivität:** $\forall x \in U : x \leq x$
2. **Antisymmetrie:** $\forall x, y \in U : x \leq y \wedge y \leq x \implies x = y$
3. **Transitivität:** $\forall x, y, z \in U : x \leq y \wedge y \leq z \implies x \leq z$
4. **Totalität:** $\forall x, y \in U : x \leq y \vee y \leq x$

Totale Ordnung

Damit Sortieren wohldefiniert ist, benötigen wir ein sinnvolles Sortierkriterium.

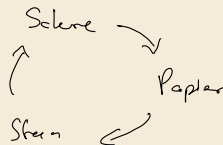
Informell: Alle Objekte sind vergleichbar und Vergleiche ergeben keine Widersprüche.

Formal: Für eine Menge U (Universum) ist eine Relation $\leq \subset U \times U$ eine *totale Ordnung*, falls folgende Bedingungen erfüllt sind

1. **Reflexivität:** $\forall x \in U : x \leq x$
2. **Antisymmetrie:** $\forall x, y \in U : x \leq y \wedge y \leq x \implies x = y$
3. **Transitivität:** $\forall x, y, z \in U : x \leq y \wedge y \leq z \implies x \leq z$
4. **Totalität:** $\forall x, y \in U : x \leq y \vee y \leq x$

Anti-Beispiele:

- Schere-Stein-Papier Gewinne ⚡ Transitivität
- $n \mid m$ (Teilbarkeit, „ n teilt m “) ⚡ Totalität



Clicker Question

Haben Sie schon einmal (beim Programmieren) eine Sortiermethode aufgerufen und explizit die gewünschte totale Ordnung mit angegeben? Falls Ja, in welcher Sprache?



A Nee, bisher nicht.

B Java

C Python

D C

E C++

F C#

G Javascript

H Perl

I sonstige



→ *sli.do/cs210*

Callbacks

Goal. Sort **any** type of data (for which sorting is well defined).

Q. How can `sort()` know how to compare data of type `Double`, `String`, and `java.io.File` without any information about the type of an item's key?

Callback = reference to executable code.

- Client passes array of objects to `sort()` function.
- The `sort()` function calls object's `compareTo()` method as needed.

Implementing callbacks.

- Java: interfaces.
- C: function pointers.
- C++: class-type functors.
- C#: delegates.
- Python, Perl, ML, Javascript: first-class functions.

Callbacks: roadmap

client

```
public class StringSorter
{
    public static void main(String[] args)
    {
        String[] a = StdIn.readAllStrings();
        Insertion.sort(a);
        for (int i = 0; i < a.length; i++)
            StdOut.println(a[i]);
    }
}
```

data-type implementation

```
public class String
implements Comparable<String>
{
    ...
    public int compareTo(String b)
    {
        ...
        return -1;
        ...
        return +1;
        ...
        return 0;
    }
}
```

Comparable interface (built in to Java)

```
public interface Comparable<Item>
{
    public int compareTo(Item that);
}
```

sort implementation

```
public static void sort(Comparable[] a)
{
    int N = a.length;
    for (int i = 0; i < N; i++)
        for (int j = i; j > 0; j--)
            if (a[j].compareTo(a[j-1]) < 0)
                exch(a, j, j-1);
            else break;
}
```

key point: no dependence
on String data type



Comparable API

Implement `compareTo()` so that `v.compareTo(w)`

- Defines a total order.
- Returns a negative integer, zero, or positive integer if v is less than, equal to, or greater than w , respectively.
- Throws an exception if incompatible types (or either is `null`).



less than (return -1)



equal to (return 0)



greater than (return +1)

`v.compareTo(w)`

$v - w \quad ? \quad 0$

Built-in comparable types. Integer, Double, String, Date, File, ...

User-defined comparable types. Implement the Comparable interface.

Implementing the Comparable interface

Date data type. Simplified version of `java.util.Date`.

```
public class Date implements Comparable<Date>
{
    private final int month, day, year;

    public Date(int m, int d, int y)
    {
        month = m;
        day   = d;
        year  = y;
    }

    public int compareTo(Date that)
    {
        if (this.year < that.year) return -1;
        if (this.year > that.year) return +1;
        if (this.month < that.month) return -1;
        if (this.month > that.month) return +1;
        if (this.day < that.day) return -1;
        if (this.day > that.day) return +1;
        return 0;
    }
}
```

only compare dates
to other dates

```
x.compareTo(y) {
    return x - y;
}
```

Comparable vs. Comparator

Für die Eingangsbeispiele wäre Comparable nicht bequem!

- ▶ Möchten oft die **gleichen** Objekte nach unterschiedlichen Kriterien sortieren
- ▶ Comparable erlaubt ja aber nur **eine** Implementierung von compareTo!

Comparable vs. Comparator

Für die Eingangsbeispiele wäre Comparable nicht bequem!

- ▶ Möchten oft die **gleichen** Objekte nach unterschiedlichen Kriterien sortieren
- ▶ Comparable erlaubt ja aber nur **eine** Implementierung von compareTo!

~> Java erlaubt alternativ, den Sortiermethoden ein **Comparator**-Objekt zu übergeben

```
1 public interface Comparator<T> {  
2     int compare(T v, T w);  
3 }
```

Gleiche Semantik:

$$c.compare(v, w) \preceq 0 \quad \equiv \quad v \preceq w \iff v - w \preceq 0$$